159.102 Instructions for Assignment 2

Assignment 2 starts in Week 5 and is due in Week 9 (**29 September 2022** at 7pm).

You should plan to complete the assignment about a week early (round about 22 September). Then you still have the week to 29 September if something unexpected happens to slow you down.

NOTE: Assignment 2 counts 20% towards your final result. The other assignments count 15% each.

It is a good idea to put your name and ID number in a comment at the top of your program.

Write a C++ program that simulates the MASSEY machine. The program must receive input in the form of a text file consisting of MASSEY machine code instructions. Your program then simulates the machine running, i.e. it works through the machine code instructions (in the correct sequence) changing values of registers and memory locations as required. You must design appropriate output that assists a machine code programmer to see what is going on with the machine code instructions. You should display items such as program counter, instructions register, values of recently changed registers, and values of recently changed memory locations. Ensure that you read through all the sections below.

Section A - input

The input to your program is a single text file containing a MASSEY machine code program.
For example, here is a typical input file where each line is a machine code instruction:

```
1102
1203
6012
40FF
E000
```

Notes about the input:

1. The programmer using the simulation can give the file whatever name they like. It is best to read in the file name when you start. A typical file name would be "prog1.txt".
2. Each line of the file is one machine code instruction. There is no other text in the file.
3. Make several (at least five) different files, each with a different machine code program. Test your program on a variety of machine code programs.
4. Do **not** attempt to check for invalid input. Assume every program contains correct machine code instructions although a program may have logic errors, e.g. instructions in the incorrect order.

(*Hint: Start off by writing a program that reads the file and displays each line on the screen.*
This is to check that your input is working correctly before you proceed to the rest of the program).

Section B – program design

Your program **must** use the following global variables:

```
int memory[256];
int reg[16];  // note: "register" is a reserved word
int pc, ir;
```

The array "`memory`" represents the memory of the MASSEY machine.
The array "`registers`" represents the registers of the MASSEY machine.
The variable "`pc`" is the Program Counter and "`ir`" is the instruction register.

The basic algorithm for your program is:

- read instructions from the file and load them into memory
- run through the instructions and execute each instruction correctly

Notes about the program design:

5. Study the MASSEY machine code instructions in the notes.
6. Ensure that your program correctly executes and valid machine code instruction.
7. You do **not** have to execute instruction 5 (floating point addition) – ignore instruction 5.
8. Do **not** check for invalid instructions. Only deal with valid instructions.
9. You must have **at least two** functions in your program.
10. **Test extensively.** Ensure that you have tested every instruction (except 5). Use machine code programs from the notes as test data.

(*Hint: get your program working for a few instructions, perhaps those in the input example. When these instructions are working correctly, expand the program to handle other instructions.*)

Section C - output

The output **must** meet the following requirements:

- display the full program (showing the memory locations) before executing the program
- identify the important items to display during execution of the instructions
- display one line for every machine code instruction (showing any changes)

For example, your display could look like this:

```
Enter the file name of the MASSEY machine code: program1.txt

Memory[0] = 1102
Memory[1] = 1203
Memory[2] = 6012
Memory[3] = 40FF
Memory[4] = E000

1102   R1 = 0002   PC = 1
1203   R2 = 0003   PC = 2
6012   R0 = 0005   PC = 3
40FF   Memory[FF] = 0005   PC = 4
Halt
```

Notes about the output:

11. Display one line of output for each machine code instruction – just after it has been executed.
12. On each line, display the current instruction and the Program Counter (which is loaded with the address of the next instruction).
13. On each line, display any registers that have changed. E.g. the first instruction above loads R1 so the value in R1 is displayed.
14. On each line, display any memory locations that have changed. E.g. the fourth instruction above loads a value into memory location FF so the value in memory[FF] is displayed.

(*Hint: to start with, get your program reading the file, loading instructions into the memory, and then displaying the memory locations and the instructions – first part of the output. Once this is working, then expand the output to display the results of the instructions.*)

Some general notes about the assignments in 159.102

- You can find the assignment instructions in a file under Assessments and also the start week.
- You submit your assignments via Stream (under Assessments) before the due date and time
- The due date and time appear on the Assignment under Assessments (where you submit)
- **Submit only your .cpp file**
- Do not submit the .exe file or any data files or screen shots of the program running
- Staff are not available to check your assignment before you submit it.
- Do not rush into submitting an assignment. You may find useful information in the notes during the week after the assignment starts.
- Assignments may use C++ knowledge from 159.101, 159.102 and elsewhere. However, if you use knowledge from elsewhere, make sure you use it correctly.

IMPORTANT rules for assignments in 159.102

- You may get assistance when writing an assignment. Assistance includes asking questions and getting ideas from teaching staff and other students. Assistance also includes asking for help when your program is not working correctly and you cannot find the error.
- You may **NOT** get someone else to write your assignment for you. If you submit a program written by someone else, you will lose a significant amount of the marks for the assignment.
- You may **NOT** copy a program from the internet. If you submit a program copied from the internet you will receive **ZERO** marks for the assignment. It is very easy for markers to find the same program on the internet.
- The important thing is that you must show that you understand what is happening in the program you submit. Teaching staff will sometimes arrange zoom sessions with students to check that they understand their submission. If this happens to you, please do not be offended – it is something we have to do as part of the quality assurance for the course.

Working on your assignments in 159.102

- You need an editor/compiler to create and run your program. Atom is provided (see notes to install Atom under Week 1) but you can use any other IDE that supports C++
- Build up your program, for example: start by only converting decimal to binary. When this is working include binary to decimal. Then build in the error checking.
- Give yourself plenty of time. Do not start 6 hours before the deadline!
- Do not give up just because the deadline arrives. You will still get some marks for a partial solution. In a difficult situation, you can apply for an extension.

Marking criteria for assignments in 159.102

Assignments are marked out of 10 and marks can be lost for:

- programs not compiling or running
- errors in code
- programs that have not been tested for a variety of situations
- programs that do not follow the instructions that are provided
- programs that appear to be written by someone else
- programs that are copied from the internet